Decision Making – Week 4

Project: Educational Microcontroller Simulator with Process Scheduling
Technology: Java **Target:** STC89C52
Week: 4 **Meeting Date:** 10-17-2026

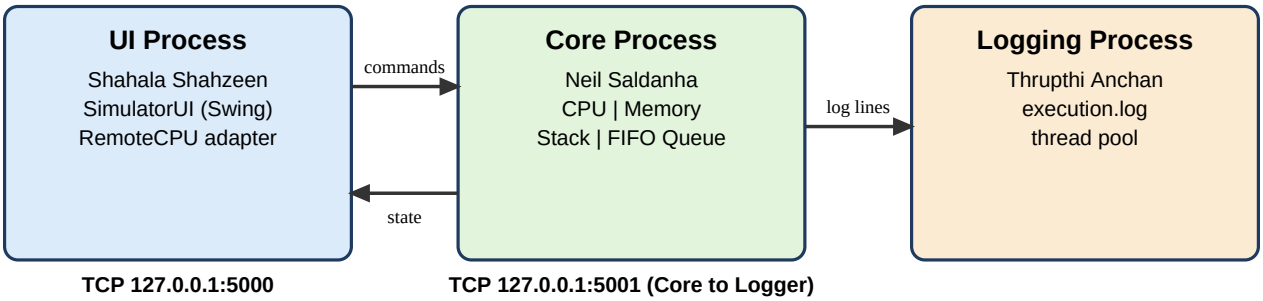
1. Purpose of Decision Making

The Week 4 decisions define how the Week 3 simulator is separated into independent processes, how those processes communicate, and how the work is divided, so that each component can be developed, tested and then integrated.

2. Decisions Made

Decision	Description
Process Separation	The simulator is split into three independent Java processes: UI (SimulatorUI), Core (CoreProcess) and Logging (LoggingProcess). The Week 3 CPU, Memory, Stack, FIFO Queue and Assembler stay unchanged inside the Core.
IPC Mechanism	Java TCP sockets on the loopback address. The UI connects to the Core on port 5000, and the Core connects to the Logger on port 5001.
Message Protocol	Line-based text. UI to Core: PING, LOADSRC (Base64-encoded source), STEP, RUN, RESET, QUIT. The Core replies with STATUS, STATE, REGS, FLAGS, MEM, STACK and QUEUE lines ended by an END line. Core to Logger: LEVEL|message lines.
UI Adapter	RemoteCPU extends the Week 3 CPU class, so the existing UI keeps working. It sends requests to the Core and mirrors the returned state.
Threading	The Core and the Logger use an ExecutorService thread pool and handle each client connection in its own thread. The Core's command processing is synchronized so CPU state is updated one command at a time. The UI uses the Swing event thread and a Swing timer.
Logging	The Core sends INFO and ERROR messages to the Logger, which adds a timestamp, appends the line to execution.log and prints it. If the Logger is unavailable, the Core continues without remote logging.
Integration	IntegrationLauncher starts the Logger, then the Core, then the UI as separate processes and stops the others when the UI closes. Windows scripts run_week4.bat and test_week4.bat build and run the system and the IPC test.
Testing	IPCIntegrationTest checks the UI/Core protocol, the Week 3 CPU execution result and the Core-to-Logger logging.
Benchmarking	Standalone and multi-process versions will be compared using the same assembly program and machine. Values are to be recorded in BENCHMARK_TEMPLATE.md.

3. IPC Architecture Diagram



Protocol: line-based text over TCP loopback sockets
UI to Core: PING, LOADSRC|<base64 source>, STEP, RUN, RESET, QUIT
Core to UI: STATUS, STATE, REGS, FLAGS, MEM, STACK, QUEUE ... END
Core to Logger: LEVEL|message (Logger prepends timestamp, appends to file)

Flow: UI → Core → CPU / Memory / Stack / Queue → Logger

IntegrationLauncher (Aman Assadi) starts Logger, then Core, then UI as separate Java processes

4. Selected IPC Mechanism and Justification

The team selected **Java TCP loopback sockets**.

Mechanism	Advantages	Drawbacks for this project
TCP sockets – **selected**	Part of standard Java (java.net); works on the Windows machines the team develops on; bidirectional, so a request and its response use one connection; supports several clients through threads; easy to test with a plain socket client.	Not a POSIX IPC primitive; adds some network-stack overhead compared with shared memory.
POSIX named pipes (FIFO)	A true POSIX mechanism; simple.	Needs Linux or WSL; one pipe per direction.
POSIX message queues	Message boundaries and priorities.	No standard Java API; would need native code.
POSIX shared memory + semaphores	Fastest.	No standard Java API; manual synchronisation.

Reasons for the selection:

- The team develops and runs the simulator on Windows, where POSIX FIFOs, message queues and shared memory are not directly available from Java.
- Sockets need no native code, so the three components can be built and tested independently by different members.
- A simple text protocol is easy to debug and to test with IPCIntegrationTest.
- The Core can serve more than one connection through its thread pool.

Note: TCP sockets are IPC but not a POSIX IPC primitive. The Week 4 brief asks for a POSIX IPC mechanism, so the team should confirm with the instructor that sockets are acceptable. If POSIX is required literally, the fallback is a POSIX FIFO implementation under Linux/WSL.

5. Reason for the Decisions

- Keeping the Week 3 engine unchanged inside the Core avoids rewriting tested code and lets the UI continue to behave as before.
- Separating logging from execution means file output is handled by its own process.
- The RemoteCPU adapter lets the existing UI be reused with minimal changes.
- A fixed set of text commands defines a clear interface between the UI and the Core.
- Launching all three processes from one launcher makes the system easy to start and demonstrate.

6. Responsibility-Based Decisions

Member	Decision / Responsibility
Shahala Shahzeen	UI process: SimulatorUI and RemoteCPU, including the connection to the Core on port 5000.
Neil Saldanha	Core process: socket server on port 5000, command handling, state replies, and the connection to the Logger.
Thrupthi Anchan	Logging process: socket server on port 5001, timestamped log lines in execution.log.
Aman Assadi	Integration: IntegrationLauncher, IPCIntegrationTest, BAT scripts, system testing, benchmarking and documentation.

7. Expected Outcome

The simulator runs as three separate processes that communicate over sockets, with each member owning one process. The remaining work is to record the benchmark values, write the performance analysis and confirm the IPC requirement.